

NalamMesh

Engineering Handover & Project Context

An offline-first public health platform for rural Maharashtra, modelled on Gadchiroli district. This document is everything a developer joining the project needs in order to work on it safely.

Problem Statement	SIH 2026 · PS 26133 — rural healthcare access and continuity
Repository	github.com/TharunS2006/HealthcareAI — working branch new
Stack	Next.js 14 (App Router, static export) · TypeScript · Zustand · IndexedDB
Scale	38 routes · 46 built pages · 9 offline stores · 7 modelled facilities
Languages	English, Hindi and Marathi — every citizen-facing screen is trilingual

Read this section first: the one rule that matters

This is a medical application. **No figure shown to a user may be invented.** Every number must come from a real record, a real computation, or be replaced with an explicit "no data" message. Several rounds of work on this codebase have been spent removing fabricated values; do not reintroduce them. Section 8 explains this in full.

1. What the project is

NalamMesh is a public health platform built for the working conditions of Gadchiroli — a heavily forested district in eastern Maharashtra where connectivity is unreliable and the nearest specialist may be hours away.

The three problems it addresses

Problem	What the software does about it
Connectivity fails. Facilities can be cut off for days.	The device is the source of truth. All reads and writes hit local IndexedDB first; syncing to other facilities happens later, if a connection appears.
Records don't travel. Paper registers stay at the facility.	A patient's triage result, vitals and history move with the referral, so the receiving tier is not starting blind.
The first contact is rarely a doctor — usually an ASHA or ANM.	An on-device triage model scores urgency, with hard clinical rules layered above it that can only escalate, never downgrade.

The four-tier care network

Indian rural public health is organised in tiers. The whole data model follows this hierarchy — referrals move *up* the tiers, and a facility's tier determines which services and entitlements apply there.

Tier	Facility	Modelled	Role
SC	Sub-Centre	2	First contact; ANM / ASHA staffed
PHC	Primary Health Centre	2	Medical Officer, basic laboratory
CHC	Community Health Centre	1	Specialists, emergency obstetric care
SDH	Sub-District Hospital	1	Surgery, inpatient beds
DH	District Hospital	1	Apex referral destination

Facility records live in `lib/data/facilities.ts` — real names, coordinates, bed capacity, staffing and services for seven Gadchiroli facilities.

2. Architecture in one page

The single most important architectural fact: **this application has no backend database**. It is a statically exported Next.js site whose data layer is the browser's own IndexedDB. Understand that and most of the design follows.

Request and data flow

```
UI component -> Zustand store -> lib/db.ts -> IndexedDB (on device)
|
optional mesh relay (Socket.io, port 3001) -> other facilities
```

Why each piece is there

Piece	Why
Static export output: 'export'	No server to be unreachable. The whole app is files that can be served from anywhere, cached by a service worker, or wrapped in the Android build.
IndexedDB via idb	Structured, indexed, durable local storage. Survives refresh and restart. Schema is versioned — currently v4 — with guarded upgrade handlers.
Zustand stores	Thin layer between UI and the database. Writes are optimistic and roll back if the persist fails, so a clinical action never silently disappears.
Service worker public/sw.js	Hand-written, no PWA package. Pre-caches 29 routes so a cold start works with no network at all. Currently v3.3.0 .
TensorFlow.js	Runs the triage model on the device. No inference request ever leaves the handset, so triage behaves identically offline.
Socket.io mesh relay	Optional . Relays records between facilities when any link exists. The app is fully functional without it.
Capacitor	Wraps the static export as an Android app for field devices.

3. Where things live

Spend ten minutes here before writing any code.

Path	Contents
app/	Next.js App Router pages — 38 routes. Citizen-facing at the root (<code>/opd</code> , <code>/queue</code>), staff screens under <code>/staff</code> .
lib/db.ts	The heart of the project. IndexedDB schema, every read and write, the audit trail. Read this file first.
lib/triage/model.ts	Triage engine: TensorFlow.js model plus the clinical override rules. See section 5.
lib/data/	Seed and reference data: facilities, hospitals, patient services and entitlements, statutory policy text.
lib/analytics/facilityMetrics.ts	Derived measures — facility scorecards, travel-time savings, mortality-prevention telemetry. All computed, none asserted.
stores/	Seven Zustand stores: patient, queue, referral, appointment, facility, auth, language.
components/gov/	Shared government-portal chrome. <code>PortalShell</code> , <code>GovPanel</code> , <code>Breadcrumb</code> , <code>PortalAside</code> , header and footer. See section 7.
public/sw.js	The service worker. Bump <code>CACHE_VERSION</code> whenever you change the pre-cache list, or clients keep the stale shell.
scripts/*.mts	Three verification suites. Run them before every commit.
server/mesh-server.ts	The optional Socket.io relay.

Conventions you'll notice

- **Trilingual by inline object.** Most pages define a local `txt` or `pageTexts` object with `en` / `hi` / `mr` keys. `lib/i18n.ts` exists but the inline pattern is dominant — follow the file you're editing.
- **Verification scripts use dynamic imports.** `lib/` is CommonJS, so the `.mts` suites use `await import()`, not static imports.
- **Dates resolve after mount.** This is a static export: computing "today" during render bakes the build date into the HTML and then mismatches on hydration. Use `useEffect`.

4. The data layer

Nine object stores (schema v4)

Store	Holds
patients	Patient demographics, ABHA id, vitals, triage status, risk flags
queue	OPD tokens — registration, call and completion timestamps
referrals	Tier-to-tier transfers with status and dispatch timestamps
appointments	Citizen bookings that convert into queue tokens
diagnostics	Test orders and results
medicineStock	Facility-level stock against the IPHS essential drug list
facilities	The seven facility records
auditLog	Every clinical change, with actor and timestamp
syncQueue	Writes awaiting relay to other facilities

Two rules enforced in lib/db.ts

1. Writes throw; they never silently no-op. If a referral cannot be found, the update raises rather than returning quietly — a silent failure in a clinical path is worse than a crash.

```
if (!ref) throw new DatabaseError(`Referral ${id} not found – status not updated to ${status}`);
```

2. Audit logging never throws. The inverse rule. If the audit write fails, the clinical action has *already* been applied — so it logs the error and carries on rather than rolling back real care.

The audit trail

- Every change to a referral, queue entry, appointment or stock line records the acting staff member and a timestamp.
- The actor is module-level state — set it with `setCurrentActor(id, role)` on login.
- **Never write patient names into audit rows.** The audit surface is readable by the District Health Officer; it stores the token number and the record id, which already resolve to the patient for anyone entitled to see it. A PII leak here was found and fixed once already.
- Viewing is restricted to MO, SPECIALIST and DHO.

5. The triage engine — read before touching

This is the highest-risk code in the project. It decides how urgently a patient is seen.

Two decision paths

Path	What it is
NEURAL_NETWORK	A TensorFlow.js model scores the case from vitals and symptoms and returns RED, YELLOW or GREEN with a confidence.
CLINICAL_OVERRIDE	27 hard rules sitting <i>above</i> the model — altered consciousness, shock-range blood pressure, critical respiratory rate, severe hypoglycaemia and others.

The safety invariant — do not break this

Overrides escalate only. A clinical rule can raise a patient's urgency. Nothing — not the model, not any later step in the flow — may lower a level that a rule has set. In a triage tool the failure that matters is the one that sends someone home. If you change this file, the escalate-only property must still hold.

How it is guarded

`npm run verify:triage` runs 20 clinical cases covering both decision paths. It must stay at 20/20. Add a case whenever you add a rule.

Priority levels

Level	Meaning	Routing
RED	Emergency	Immediate care or transfer to the highest reachable tier
YELLOW	Urgent	Seen the same day; escalate if vitals move
GREEN	Routine	Standard OPD flow with follow-up scheduled

6. The modules

Nine functional modules, all writing to the same local database — which is why a patient seen at a sub-centre is already known at the district hospital.

Module	Route	What it does
OPD Intake & Triage	/opd	Registration, ABHA capture, vitals entry, runs triage, issues a token
District Command	/dashboard	Indicators, facility network status, high-risk action list
OPD Queue	/queue	Live token board; wait times computed from real call timestamps
Referrals	/referrals	Tier-to-tier transfer with live transit tracking
Appointments	/appointments	Citizen booking; slot capacity derived from actual facility staffing
Medicine Stock	/medicine	IPHS drug list, low-stock and near-expiry alerts
Diagnostics	/diagnostics	Test orders and results against the patient record
High-Risk Follow-up	/followup	Scheduled recall for ANC, SAM and chronic cases
Teleconsult	/teleconsult	Specialist link-up. Labelled a simulated preview — there is no WebRTC yet.
Facilities & Services	/facilities, /services-info	Directory, entitlements, scheme eligibility (PMJAY gated to CHC and above)
Audit Trail	/audit	Restricted to MO, Specialist and DHO

Seven staff roles scope what each user sees: ASHA, ANM/CHO, Medical Officer, Specialist, Pharmacist, Lab Technician, District Health Officer.

7. UI conventions

The interface deliberately imitates an Indian government department portal — NIC built, GIGW 3.0 compliant. It is modelled on india.gov.in, mohfw.gov.in and arogya.maharashtra.gov.in. This is a requirement, not a stylistic preference.

Do

- Square corners. Radius 0–2px. Portals are built from bordered boxes.
- Separate content with a **1px rule**, never a drop shadow. All elevation is flattened globally in `tailwind.config.ts`.
- Put content in a `GovPanel` — a solid navy heading bar above a square bordered body.
- Present data in **tables** with a header row and alternating row shading.
- Keep type small and dense: body 12–13px, table text 12px, labels 11px.
- Use `PortalShell` for the three-column layout.

Do not — these read as machine-generated

- Coloured left accent rails on cards (the rotating blue/orange/green stripe). These were removed from seven files; do not bring them back.
- Large KPI tiles with oversized display numbers. Use a bordered indicator table.
- Pill badges carrying a checkmark glyph, such as "Verified" or "Ready".
- Rounded cards with soft shadows in a uniform three-across grid.
- Icons inside coloured circles.
- Hover elevation.
- Emoji, anywhere.

Layout trap that has already bitten once

Sidebar is a fixed `w-64` with `flex-shrink-0`, because most pages still use the legacy flex layout. `PortalShell` therefore uses **explicit grid tracks** — `16rem / minmax(0, 1fr) / 20rem` — not 12-column fractions. Changing the sidebar to `w-full` makes it swallow the whole row on every unmigrated page. Two pages use `PortalShell`; thirteen still use the flex layout.

8. The no-invented-data rule

The most important convention in this codebase, and the one most easily broken by accident.

It is tempting, when a demo looks empty, to pad a number so the screen looks populated. In a medical tool that is not a cosmetic choice — it puts a figure in front of a clinician that nothing backs. Every one of the patterns below was found in this codebase and removed.

Anti-pattern	Why it is wrong	Do instead
<code>patients.length + 152</code>	Inflates a real count with an invented offset	Show the real count, even if it is 5
<code>count > 0 ? count : '4'</code>	Falls back to a fake number when the truth is zero	Show zero — zero is information
78% Occupied hardcoded	Asserts a measurement nobody took	Compute from facility bed records
"Avg response: 28 min"	A plausible-looking figure with no source	Derive from timestamps, or omit
Hardcoded dated notices	Stale the day after they are written	Compute forward from today

What good looks like

- **Bed occupancy** sums `beds.total` and `beds.occupied` across facility records, and renders "Not reported" when no facility has recorded capacity.
- **Queue wait times** are computed from `calledAt - registeredAt`. Where nobody has been called yet, the board says so instead of showing a placeholder.
- **Announcements** list the next occurrence of each recurring clinic day, computed from `nextOccurrence()` — so the dates move on their own.
- **Referral transit clocks** only run where a real dispatch timestamp exists. A referral with no dispatch shows no clock rather than a fabricated elapsed time.

Rule of thumb

If you cannot point at the record a number came from, it does not go on the screen. State a real zero, or say "no data".

9. Running and verifying it

Local development

```
git clone https://github.com/TharunS2006/HealthcareAI.git
cd HealthcareAI && git checkout new
npm ci
npm run dev # app on :3000
npm run dev:full # app on :3000 + mesh relay on :3001
```

Before every commit

```
npx tsc --noEmit
npm run verify:triage # 20/20 clinical cases — must not drop
npm run verify:metrics # facility scorecard calculations
npm run verify:services # entitlements and clinic schedules
npm run build # must produce 46 pages
```

Testing offline properly

Do not trust DevTools' "Offline" toggle — during this project it left `navigator.onLine` `true` and requests still succeeded. **Kill the server instead.** Build, serve the `out/` directory, load the app, stop the server, then navigate. That is how the offline claims were verified: a route never visited since install rendered from cache, and a clinical write persisted with audit rows going from 1 to 3.

Gotchas that cost time

Symptom	Cause and fix
A route 404s in dev but is fine in the build	Stale dev route manifest, usually after <code>npm run build</code> wiped <code>.next</code> under a running dev server. Stop dev, <code>rm -rf .next</code> , restart.
Pages render unstyled	Same cause — the build removed the dev server's assets.
Offline start misses a new page	You added a route without adding it to <code>PRECACHE_ROUTES</code> and bumping <code>CACHE_VERSION</code> in <code>public/sw.js</code> .
Hydration mismatch on a date	A date computed during render in a static export. Move it into <code>useEffect</code> .
Sidebar swallows the page	See the layout trap in section 7.

10. Where the project stands

Verified working

Check	Status
Production build	Passes — 46 static pages
TypeScript	Clean, no errors
Triage clinical suite	20 / 20, both decision paths exercised
Facility metrics suite	Passing
Services & entitlements suite	Passing
Offline app shell	29 routes pre-cached, verified with the server stopped
Console errors	None across all 38 screens

Known gaps — be honest about these

- **Teleconsult has no WebRTC.** It is a simulated preview and is labelled as such in the UI. Do not describe it as working video consultation.
- **The UI rework is partial.** Two pages use `PortalShell`; thirteen still use the legacy flex layout. They have picked up the global token changes and lost their accent rails, but their card grids have not yet become tables.
- **The seven statutory policy pages are English-only** while the rest of the portal is trilingual. Defensible — most state portals do the same — but inconsistent.
- **No Vercel deployment is linked.** The repository is not connected to a Vercel project, so pushes do not deploy.

Suggested first tasks for a new developer

- Read `lib/db.ts` end to end, then `lib/triage/model.ts`.
- Run all three verification suites and make sure you understand what each asserts.
- Migrate one legacy page to `PortalShell` — `/queue` is a good first one — and convert its card grid to a table.
- Do the offline test yourself, by killing the server. It is the clearest way to understand what the architecture actually buys.

If you remember three things

1. The device is the source of truth — there is no backend. **2.** Triage overrides escalate only. **3.** Never put a number on screen that you cannot trace to a record.